

The MediaTek logo is an orange parallelogram with the word "MEDIATEK" in white, uppercase, sans-serif font.

MT8676 SCP

Version: 1.01
Release date: 2026-04-02

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	日期
0.1	2026-04-02	Benjamin Lin	Initial draft

目录

版本记录	2
目录	3
图片目录	5
表格目录	5
1 概述	7
1.1 硬件架构	7
1.2 软件架构	9
1.3 小结	10
2 源代码树	11
2.1 LK 引导加载程序	11
2.1.1 Linux 内核驱动	11
2.2 FreeRTOS Tree	11
3 构建系统	12
3.1 配置文件	12
3.1.1 LK 引导加载程序	12
3.1.2 Linux 内核驱动	12
3.1.3 FreeRTOS	12
编译命令	13
3.2 13	
3.3 镜像布局	13
4 启动顺序	14
5 处理器间中断	15
5.1 SCP 端的使用	16
5.2 内核端的使用	22
6 DRAM 访问流程	24
6.1 在 Linux 内核中保留内存	24
6.1.1 在共享内存中注册用户	25
6.1.2 扩展共享内存	26
6.2 通过 ID 获取保留内存	26
6.3 在 SCP 端获取保留的 DRAM 区域	27
6.4 将 DRAM 地址从 AP 视图重映射到 SCP 视图	27

6.5	请求系统总线和 DRAM	28
7	驱动程序指南	29
7.1	驱动程序初始化	29
7.2	添加新驱动程序	29
7.3	中断	30
7.3.1	IRQ 注册	30
7.3.2	使能中断	31
7.3.3	禁用 IRQ	31
7.3.4	注册唤醒源	32
7.4	锁	33
7.5	DMA	33
7.6	硬件信号量	34
7.7	GPIO 和 EINT	36
7.7.1	GPIO 使用	37
7.7.2	EINT 使用	38
8	调试方法	39
8.1	PRINTF_* 使用	39
8.2	串口	39
8.3	异常日志分析	40
8.3.1	Trace Buffer	42
8.4	核心转储	43
8.4.1	LLDB 基本命令	45
8.5	性能评估和运行记录	48
8.5.1	性能评估	48
8.5.2	运行记录	49
9	地址检测器	56
9.1	ASAN 使用方法	56
9.2	如何将代码移动到 DRAM	58
10	附录	60
10.1	常见问题及解决方法	60
10.1.1	Malloc 失败	60
10.1.2	非对齐访问	61
10.2	代码大小限制	62

10.2.1 代码大小检查工具的使用	62
10.3 Scp_Region_Info 架构	63
10.4 SCP 恢复机制	64
10.4.1 SCP 恢复流程	64
10.4.2 恢复通知流程	65
附件一 附加条款	67

图片目录

图 1-1. MTK 23P SCP 架构	8
图 1-2. MTK MT8676 SCP 软件架构	9
图 4-1. SCP 启动流程框图	14
图 5-1. Tinysys IPI 通用架构	15
图 8-2. 串口工具设置	40
图 8-5. 性能评估日志示例	49
图 8-6. 飞行记录控制台模式示例	51
图 8-7. 操作方法	52
图 8-8. 运行记录的 GUI 显示模式	53
图 8-9. 详细信息显示	53
图 8-10. 任务被唤醒的事件	54
图 8-11. 队列事件的示例	54
图 8-12. 队列事件的详细信息	54
图 8-13. 软件定时器示例	54
图 8-14. 临界区示例	55
图 8-15. 中断服务程序示例	55
图 8-16. OS 时钟节拍示例	55
图 10-1. 内存工具报告	62
图 10-2. SCP 重置流程	64

表格目录

表 1-1. 硬件规格表	7
表 1-2. 架构规格表	10

表 6-1. 标准 DRAM 访问流程	24
表 6-2. 默认重映射规则表	28
表 7-1. EINT 和 GPIO 的映射引脚名称	36
表 7-2. GPIO 控制寄存器表	37
表 8-1. PRINTF 使用场景	39
表 8-2. UART 引脚名称	39
表 8-3. 编译器选项和定义	48
表 8-4. 编译器选项	49
表 8-5. Flight Record 定义	50
表 9-1. 编译器选择和定义	56
表 9-2. 异常类型表	56

1 概述

系统协处理器（SCP）是一个子系统，设计用于在系统处于低功耗状态时执行始终开启的任务。

1.1 硬件架构

SCP 由专用处理器、SRAM、DMA 和外设（如 I2C、GPIO）组成。

- 处理器：SoC 内嵌 MDSP RV55NN x 1，每个核心有 2 个硬件线程，采用 RISC-V 架构
 - 单精度浮点
 - 压缩指令
 - 用于语音加速的 DSP ISA
 - 内部 NN 引擎用于矢量指令
 - 每个核心独立的指令缓存 32K，数据缓存 32K
 - 工作频率从 266 到 800MHz
- 内存：2MB TCM
- DMA 吞吐量：
 - 从 TCM 到 DRAM 的数据传输：63.8 MB/s
 - 从 DRAM 到 TCM 的数据传输：39.9 MB/s
- 外设：
 - I2C x 1, I3C x 6
 - SPI x 4
 - UART x 2
- 详细信息

表 1-1. 硬件规格表

名称		MT6897
Core		RV55NN x 1 (每核 2 个硬件线程)
Cache	L1\$	I\$/ D\$: 32KB/32KB
	L2\$	256KB
TCM	L1TCM	NA
	L2TCM	2MB
外设		- I2C x 1 - I3C x 6 - SPI x 4 - UART x 2
DMA		8 channels (0&1 reserved for I2C)
VoW I/F		3-mic

名称	MT6897
Operating Frequency	800MHz@Vscp 0.75V
Performance (CoreMark@Vmin)	TBD
Power Efficiency (CoreMark/mW)	TBD

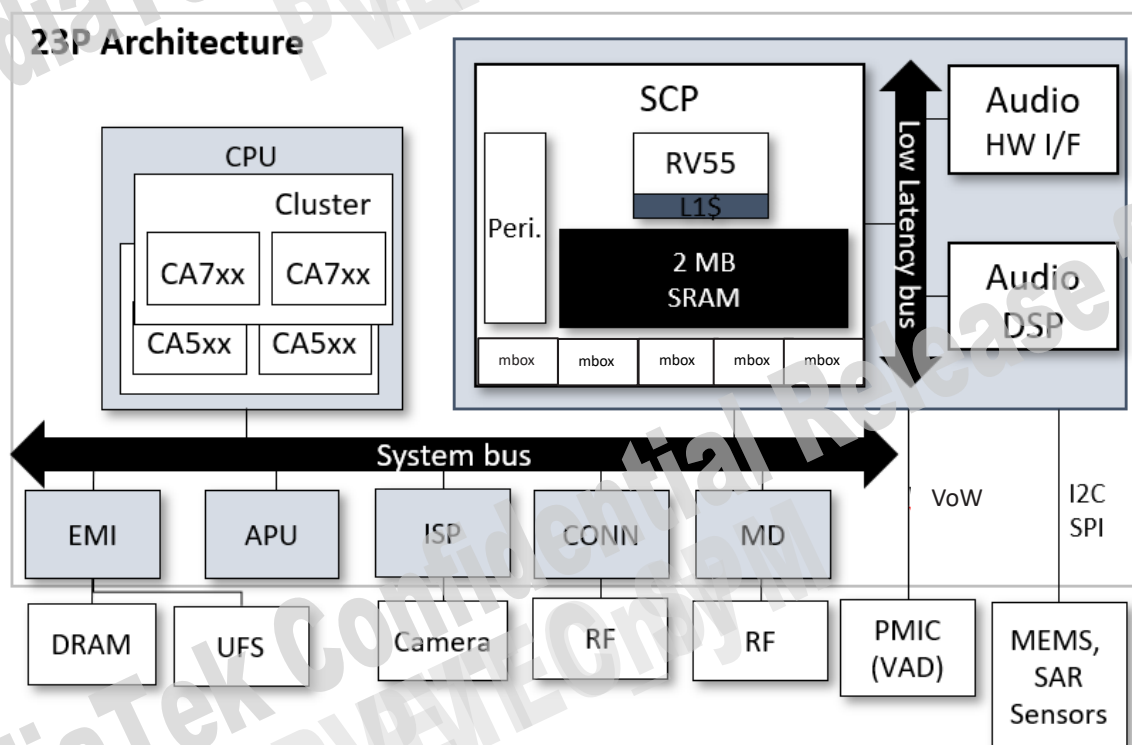


图 1-1. MTK 23P SCP 架构

1.2 软件架构

SCP 软件架构基于 AWS FreeRTOS v10.1.0，这是一个支持多任务、互斥、信号量和软件定时器的实时内核。软件包还包括音频和传感器集线器等中间件，但本文档不讨论这些细节。

SCP 基于 OpenAMP rpmsg/remoterproc 框架的 IPI（处理器间中断）与 Linux 内核通信。我们还提供了其他机制以实现 AP 和 SCP 之间的协作：

- IPI（处理器间中断）– 详见第 5 章
- 硬件信号量 – 详见第 7.6 节
- 日志记录– 详见第 [Error! Reference source not found.](#) 节
- SCP 恢复机制 – 详见第 10.4 节

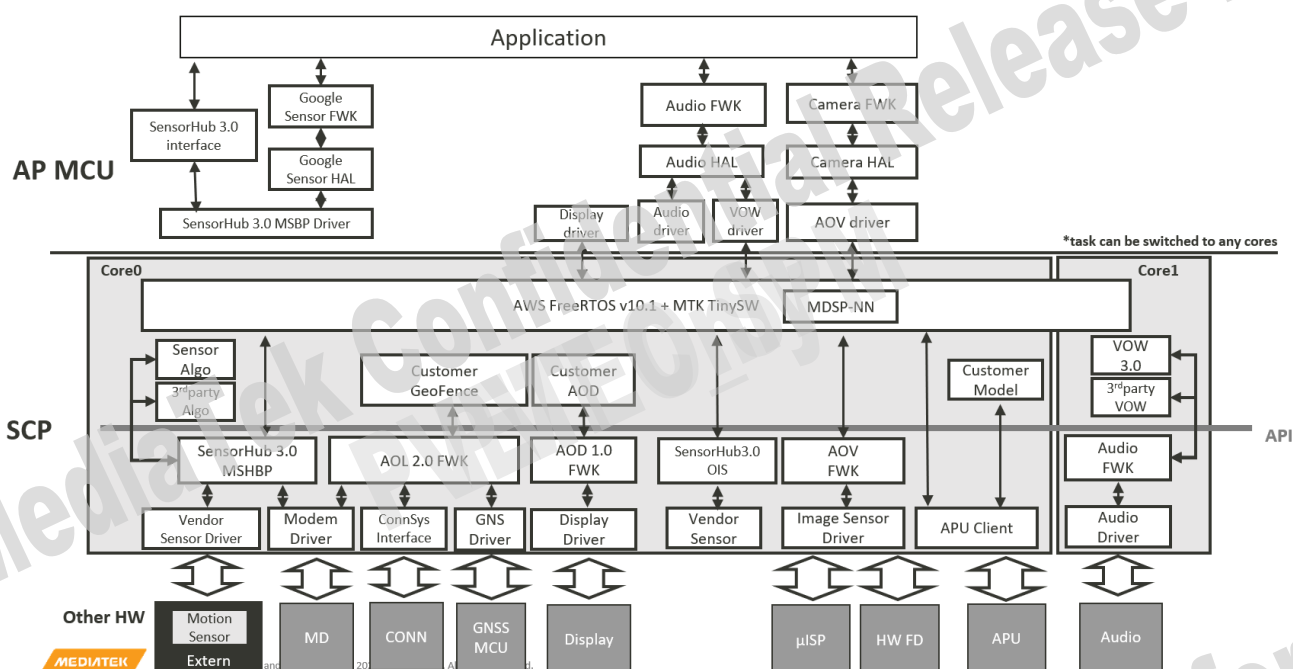


图 1-2. MTK MT8676 SCP 软件架构

1.3 小结

表 1-2. 架构规格表

Item	Value
Platform	Mt8676
Project	\$PROJECT
Linux version	6.1
FreeRTOS version	10.1.0.1
ISA	RV55-NN
L2 TCM	2MB

2 源代码树

SCP 源代码树包括引导加载程序（即 LK）、Linux 内核和 FreeRTOS。列举如下：

2.1 LK 引导加载程序

- src/bsp/lk2/platform/mediatek/common/scp/scp.c
- src/bsp/lk2/platform/mediatek/common/scp/RV/
- src/bsp/lk2/platform/mediatek/mt8676/include/platform/scp_plat_soc.h

2.1.1 Linux 内核驱动

- SCP driver path

master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp

- SCP DTS path

master/src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/Mt8676.dts

2.2 FreeRTOS Tree

- RTOS Kernel

master/src/tinysys/mt8676/kernel/FreeRTOS_v10.1.0.1

- Platform and peripheral drivers

master/src/tinysys/mt8676/scp

master/src/tinysys/mt8676/common

- Libraries

master/src/tinysys/mt8676/scp/middleware

- Toolchain

prebuilts/devtool/clang/md32rv/linux-x86/CodeLine_212

3 构建系统

在编程之前，开发人员可能想知道如何使用基本的 SCP 配置进行编译。本章将展示配置文件的位置和如何通过命令生成 SCP 镜像。

3.1 配置文件

3.1.1 LK 引导加载程序

```
src/bsp/lk2/platform/mediatek/mt8676/rules.mk
```

- `MODULES_DEP += platform/$(PLATFORM)/common/scp/RV`

3.1.2 Linux 内核驱动

```
master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp/rv/
```

- 启用或禁用 SCP 驱动程序: `CONFIG_MTK_TINYSYS_SCP_SUPPORT`
- 切换功能，例如语音唤醒和传感器中心等功能的开关

3.1.3 FreeRTOS

```
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/platform.mk
```

- 平台的默认配置额外的 `CFLAGS`
- 额外的 `LDFLAGS`

驱动程序/中间件的 `c` 对象文件和包含路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/project.mk
```

- 项目配置

通过在 `platform.mk` 文件中覆盖选项来自定义项目

3.2 编译命令

```
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
source meta/poky/oe-init-build-env
bitbake tinysys-scp -c cleansstate && bitbake tinysys-scp 2>&1 | tee yocto_scp.log
```

Build Result scp.img path:

- build\tmp\work\aarch64-poky-linux\tinysys-scp\1.0\deploy-tinysys-scp
- build\tmp\work\aarch64-poky-linux\tinysys-scp\1.0\image\lib\firmware

3.3 镜像布局

- EMMC/UFS 存储器中存在的两个分区：scp1 和 scp2
 - scp1: main and active partition
 - scp2: backup for AB system
- Image: scp.img:
 - tinysys-scp-RV55_A.bin: firmware/data located in SRAM
 - tinysys-scp-RV55_A.elf: elf with symbol, for debug purpose
 - tinysys-scp-RV55_A_DRAM.bin: firmware/data located in DRAM

4 启动顺序

在 SCP 镜像准备好之后，我们需要了解镜像是如何加载到 SRAM/DRAM 中以及 SCP 是如何启动运行的。整个流程由 LK 引导加载程序、Linux 内核和 SCP 固件共同完成，具体如下：

- LK 引导加载程序：(yocto/src/bsp/lk2/platform/mediatek/common/scp/scp.c)
 - 为 SCP 镜像分配永久性 DRAM 内存加载/验证 SCP 镜像设置 EMI MPU (AP 只读)
- 内核
 - 初始设置(mbox/ipi/logger/...)
 - 启动 SCP
- SCP
 - 加载程序：(master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/boot55.S)
 - 将 SCP 镜像加载到 SRAM 跳转到 FreeRTOS
 - FreeRTOS (master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/src/main.c)
 - 驱动程序初始化设置 MPU

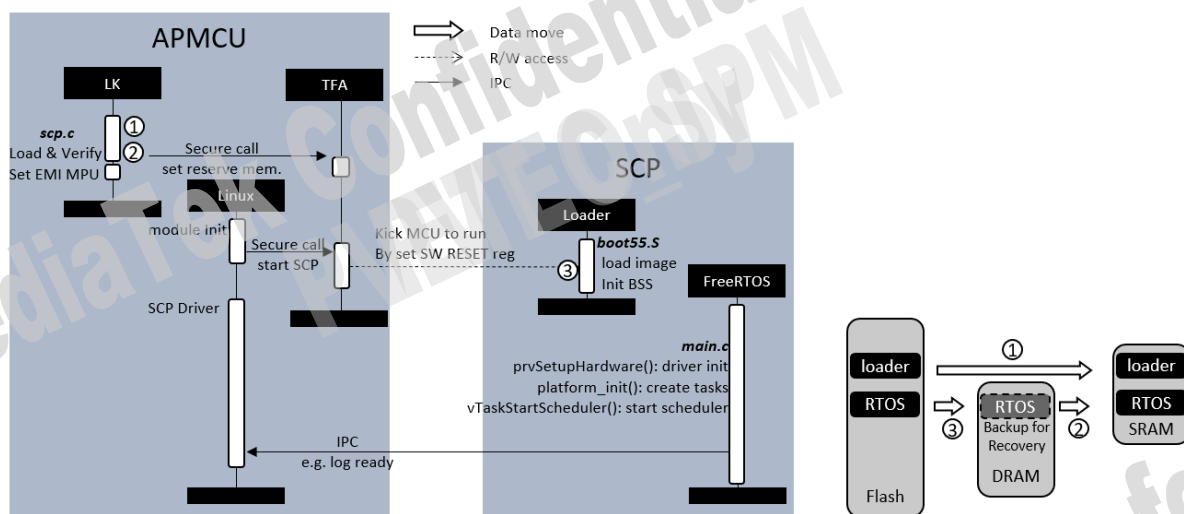


图 4-1. SCP 启动流程框图

5 处理器间中断

处理器间中断 (IPI) 是一种在 Linux 和 FreeRTOS 驱动程序之间传递消息的机制。它包括以下部分：

1. 一块共享内存：用于交换数据
2. 一组中断：用于相互通知

图 5-1 展示了 Tinysys IPI 架构。软件架构由四层组成：

1. Synchronization layer: 用于 AP 和 Tinysys 之间通信的公共 API
2. Rpmmsg layer: 提供阻塞/非阻塞发送功能
3. Queue layer: 进行队列操作和管理功能
4. Physical layer: 进行物理硬件操作

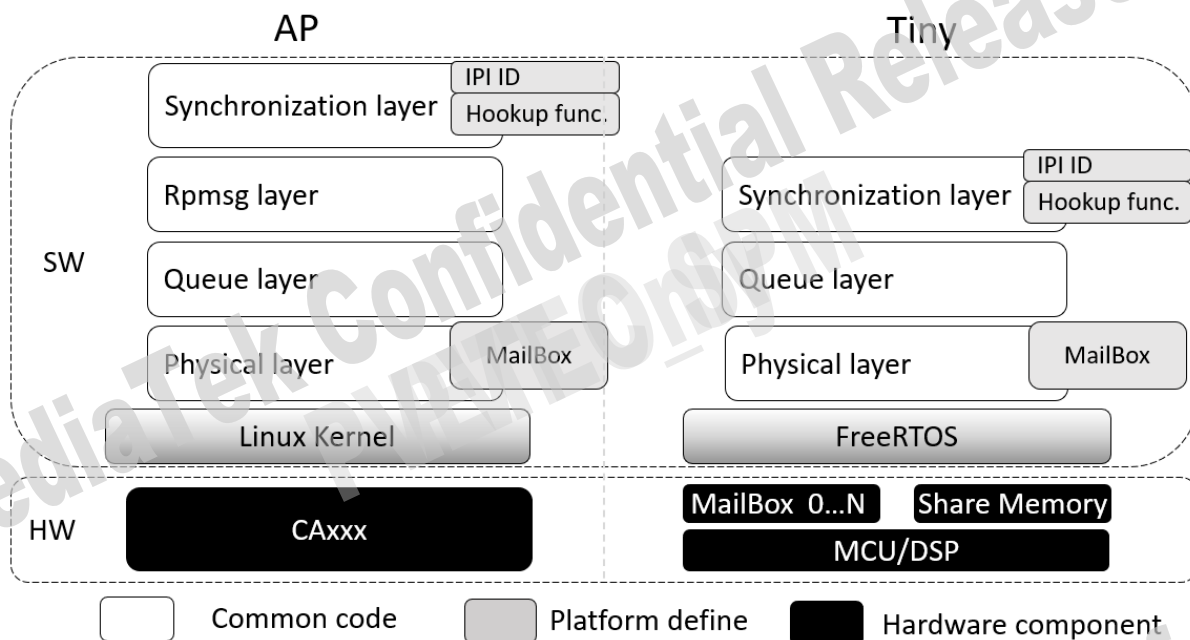


图 5-1. Tinysys IPI 通用架构

邮箱 (MailBox) 是预定义的。开发人员使用以下 API 将 IPI 发送到 Tinysys，包括 IPI ID 和注册回调函数 (ipi_cb)。

5.1 SCP 端的使用

i. 在 `ipi_id.h`, `ipi_table.h` 和 `mbox_pin.h` 中添加新的 IPI ID 信息。发送和接收信息由不同的 IPI 表描述。

- 文件描述

- `ipi_id.h`: 定义 IPI ID (必须唯一)
- `ipi_table.h`: 定义 IPI 引脚表 (引脚槽总数不能超过邮箱槽)
- `mbox_ipi.h`: 定义 IPI 引脚槽

- 路径

```
master/src/tinysys/mt8676/scp/drivers/RV55_A/Mt8676/mbox/ipi_id.h
master/src/tinysys/mt8676/scp/drivers/RV55_A/Mt8676/mbox/ipi_table.h
master/src/tinysys/mt8676/scp/drivers/RV55_A/Mt8676/mbox/mbox_pin.h
```

- IPI_TABLE (发送器引脚说明)

```
/* mbox pin structure, this is for send definition,
 * ipi=endpoint=pin
 * mbox : (mbox number)mbox number of the pin, up to 16
 * offset : (slot)msg offset in share memory, 4 bytes alignment, up to 1024*4 KB
 * send_opt : (opt)send opt, 0:send ,1: send for response
 * lock_opt : (opt)option 0: mutex, 1: busy wait
 * msg_size : (slot)message size in words, 4 bytes alignment
 * pin_index : (bit offset)pin index in the mbox
 * ipi_id : (ipi_id)ipi_id in the mbox
 * mutex : (mutex)mutex for remote response
 * sema_ack : (sema_ack)completion for remote response
 * send_record : send pin record information
 */
struct pin_send {
    unsigned int mbox:4,
                offset:20,
                send_opt:2,
                lock_opt:2;

    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t mutex;
    SemaphoreHandle_t sema_ack;
    struct mtk_mbox_send_record send_record;
};
```

- IPI_TABLE (接收器引脚说明)

```

/*
 * mbox pin structure, this is for receive definition,
 * ipi=endpoint=pin
 * mbox:                (mbox number)mbox number of the pin, up to 16
 * offset:              (slot)msg offset in share memory, 4 bytes alignment, up to
1024*4 KB
 * recv_opt:            (opt)recv option, 0:receive ,1: response
 * lock_opt:            (opt)option 0: mutex, 1: busy wait
 * buf_full_opt:        (opt)buffer option 0:drop, 1:assert, 2:overwrite
 * cb_ctx_opt:          (opt)callback option 0:isr context, 1:process context
 * msg_size:            (slot)msg used slots in the mbox, 4 bytes alignment
 * pin_index:           (bit offset)pin index in the mbox
 * ipi_id:              (ipi_id)ipi_id in the mbox
 * notify:              (completion)notify process
 * mbox_pin_cb:         (cb)cb function
 * pin_buf:             (void*)buffer point
 * prdata:              (void*)private data
 * recv_record:         receive pin record information
 */
struct pin_recv {
    unsigned int mbox:4,
                offset:20,
                recv_opt:2,
                lock_opt:2,
                buf_full_opt:2,
                cb_ctx_opt:2;

    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t notify;
    void (*mbox_pin_cb) (unsigned int id, void *prdata, void *data, unsigned int len);
    void *pin_buf;
    void *prdata;
    struct mtk_mbox_recv_record recv_record;
};

```

- 示例：在 `ipi_id.h` 中添加新的 IPI ID

```
##Add IPI_NEW_ID at ipi_id.h
enum {
    .
    .
    IPI_NEW_ID =39,
    IPI_COUNT
};
```

- 示例：在 `mbox_pin.h` 中添加新的 PIN 大小定义

```
##Add PIN_IPI_NEW_ID_SIZE at mbox_pin.h

#define PIN_IPI_NEW_ID_SIZE 2
```

- 示例：在 `ipi_table.h` 中添加新的引脚设置

- 以下指南不得违反：

- 在同一个邮箱（mbox）中，发送表和接收表的总消息大小（msg_size）不得超过 64。
- 新的引脚设置必须与相同的邮箱编号捆绑在一起。

```
##Add new pin setting for send or rcv table at ipi_table.h
```

```
Struct pin_send mbox_pin_send_table [] = {
    {0, 0, 0, 0, ..., {}},
    .
    .
    {0, 0, 0, 0, PIN_IPI_NEW_ID_SIZE, 0, IPI_NEW_ID, 0, 0, {}},
    {1, 0, 0, 0, ..., {}},
    .
    .
    {2, 0, 0, 0, ..., {}},
    .
    .
}
```

ii. 注册 IPI 处理程序

```
int ipi_register(unsigned int ipi_id, void *cb, void *prdata, void *msg)
```

Description

To register IPI handler

Parameters

id: id declared in ipi_id.h

cb: IPI handler, a callback

prdata: IPI handler parameter, for customized

msg: msg buffer, data recv from AP

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

BUSY: the channel is busy. Need to retry.

• 示例:

```
#include "scp_ipi.h"
```

```
int ret;
```

```
ret = ipi_register(IPI_NEW_ID, ipi_cb, 0, "ipi_cb name");
```

```
if (ret != IPI_ACTION_DONE)
```

```
    PRINTF_E("Register IPI failed\n");
```

```
void ipi_cb(int id, void *prdata, void *data)
```

```
{
```

```
    /* data: received message from kernel */
```

```
    unsigned int rcv_data = *(unsigned int *)data;
```

```
    ...
```

```
}
```

iii. 接收 IPI

```
int ipi_recv(unsigned int ipi_id)
Description
    API for apps to receive an IPI from AP
Parameters
    id: IPI id declared in ipi_id.h
Return values
    DONE: complete successfully
    ERROR: something wrong in lower layer driver, i.e. mbox
```

• 示例:

```
#include "scp_ipi.h"

static void xxx_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_recv(IPI_NEW_ID);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

```
int ipi_recv_reply(unsigned int ipi_id, void *reply_data, int len)
Description
    API for apps to receive an IPI from AP and reply
Parameters
    id: IPI id declared in ipi_id.h
    reply_data: the message which will be replied to Linux kernel
    len: message length(4 bytes as a unit)
Return values
    DONE: complete successfully
    ERROR: something wrong in lower layer driver, i.e. mbox
```

• 示例:

```
#include "scp_ipi.h"

static void new_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_recv_reply(IPI_NEW_ID, (void *)&reply_data, reply_data_len);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

iv. 发送 IPI

```
int ipi_send(unsigned int ipi_id, void *data, int len, unsigned long retry_timeout)
```

Description

API for apps to send an IPI to AP

Parameters

id: IPI id declared in ipi_id.h

data: the message which will be sent to Linux kernel

len: message length(4 bytes as a unit)

retry_timeout: the times of retry

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

• 示例:

```
#include "scp_ipi.h"

ipi_status ret;
ret = ipi_send (IPI_NEW_ID, (void *)&msg, msg_size, 0, IPI_SCP2AP);
if (ret != DONE)
    PRINTF_E("Send IPI failed\n");
```

v. IPI 状态

如果在运行时使用过程中出现任何问题，请首先检查 API 的返回值，返回值参考如下表所示：

#define IPI_ACTION_DONE	0	
#define IPI_ILLEGAL	-1	/* illegal ipi index */
#define IPI_DUPLEX	-2	/* the ipi has be registered */
#define IPI_UNAVAILABLE	-3	/* can't find this ipi pin define */
#define IPI_NO_MSGBUF	-4	/* ipi receiver doesn't has message buffer */
#define IPI_NO_MEMORY	-5	/* the message length is large than defined */
#define IPI_PIN_BUSY	-6	/* send message timeout */
#define IPI_RECV_TIMEOUT	-7	/* receive message timeout */
#define IPI_MBOX_ERR	-99	/* some error from physical layer */

5.2 内核端的使用

i. 在 `scp.h` 和 `mt6897.dts` 文件中添加新的 IPI ID 和 pin table

- 文件描述

- `scp.h`: 定义 IPI (ID 必须与 SCP 端同步)
- `mt6897.dts`: 定义引脚表 (引脚表必须与 SCP 端同步)

- 路径

`master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp/include/scp.h`
`master/src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/mt6897.dts`

在设备树文件 (DTS) 中描述 SCP IPI 的设置, 以确保系统的正常运行和通信。

- 示例: 在 `scp.h` 中添加新的 IPI ID 和消息大小

```
## Add new IPI ID and message size at scp.h
```

```
.
.
.
#define PIN_IPI_NEW_ID_SIZE      2
.
.
.
enum {
.
.
    IPI_NEW_ID = 39,
    SCP_IPI_COUNT
}
```

- 示例: 在 `mt6897.dts` 中添加新的引脚设置:

```
scp: scp@1cb00000 {
.
.
    send_table =
        // <id, mbox, send_size>
.
.
    <...>;
    recv_table =
        //< id, mbox, recv_size, recv_opt>,
        < 39, 0, 2, 0>,
.
.
    <...>;
}
```

ii. 注册一个 IPI

```
int mtk_ipi_register(struct mtk_ipi_device *ipidev, int ipi_id, mbox_pin_cb_t cb, void *prdata, void *msg)
```

Description

To register IPI handler

Parameters

ipidev: ipidev declared in scp.h

ipi_id: id declared in scp.h

cb: IPI handler, a callback

prdata: IPI handler parameter, for customized

msg: msg buffer, data recv from SCP

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

iii. 发送一个 IPI

```
int mtk_ipi_send(struct mtk_ipi_device *ipidev, int ipi_id, int opt, void *data, int len, int timeout)
```

Description

API for apps to send an IPI to scp

Parameters

ipidev: ipidev declared in scp.h

ipi_id: IPI id declared in scp.h

opt: IPI_mode, IPI_SEND_WAIT or IPI_SEND_POLLING, should be sync with pin table

data: the message which will be sent to SCP

len: message size(4 bytes as a unit)

timeout: busy wait, as ms per unit

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

SCP_NOT_READY: scp is not ready. Need to retry.

iv. 接收一个 IPI

```
int mtk_ipi_recv(struct mtk_ipi_device *ipidev, int ipi_id)
```

Description

API for apps to receive an IPI from scp

Parameters

ipidev: ipidev declared in scp.h

ipi_id: IPI id declared in scp.h

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

SCP_NOT_READY: scp is not ready. Need to retry.

6 DRAM 访问流程

由于无论 SCP 的状态如何，DRAM 和系统总线都可能被关闭，SCP 开发人员必须遵循以下步骤以避免系统挂起。以下是 AP 和 SCP 之间通过 DRAM 共享信息的概述。

(注: 请参考章节 6.3 以获取 SCP 端保留的 DRAM 地址)

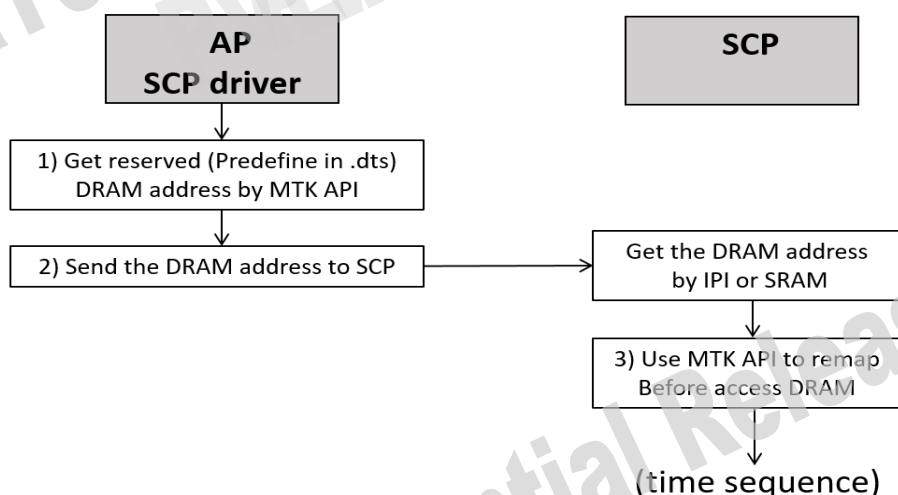


表 6-1. 标准 DRAM 访问流程

#	步骤	备注
0	检查/添加预定义表 <code>scp_reserve_mblock[]</code>	
1	SCP Linux 驱动通过 API 获取保留的物理地址	<code>phys_addr_t scp_get_reserve_mem_phys(scp_reserve_mem_id_t id)</code> <code>phys_addr_t scp_get_reserve_mem_virt(scp_reserve_mem_id_t id)</code> <code>phys_addr_t scp_get_reserve_mem_size(scp_reserve_mem_id_t id)</code>
2	SCP Linux 驱动通过 IPI 将地址发送到 SCP	<code>scp_ipi_send(uint32 id, void* buf, uint len)</code>
3	SCP 通过 API 访问 DRAM	<code>uint32_t ap_to_scp(uint32_t ap_addr)</code>
4	在使用前启用 DRAM 资源，在使用后禁用 DRAM 资源	<code>void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id)</code> <code>void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id)</code>

6.1 在 Linux 内核中保留内存

为了在 AP 和 SCP 之间交换数据，需要在 DRAM 中保留一个空间。首先，我们需要在 `scp_reserve_mblock[]` 中添加条目。这些定义可以在如下路径找到：

- 路径:

```

{lk2_repo}/platform/mediatek/mt6897/rules.mk
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
  
```

```
{kernel_repo}/drivers/misc/mediatek/scp/include/scp.h
{kernel_repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h
```

6.1.1 在共享内存中注册使用者

步骤1. 在 `scp_reserve_mem_id_t` 中添加新的 ID

```
{kernel_repo}/drivers/misc/mediatek/scp/include/scp.h
enum scp_reserve_mem_id_t {
    SCP_A_SECDUMP_MEM_ID = 0,
    VOW_MEM_ID,
    SENS_MEM_ID,
    ...
    USER_MEM_ID, // 在此处添加
    NUMS_MEM_ID,
}
```

步骤2. 在 `scp_reserve_mblock` 数据结构中添加成员

```
{kernel_repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h
static struct scp_reserve_mblock scp_reserve_mblock[] = {
    {
        .num = SCP_A_SECDUMP_MEM_ID,
        .start_phys = 0,
        .start_virt = 0,
        .size = 0,
    },
    {
        .num = VOW_MEM_ID,
        .start_phys = 0x0,
        .start_virt = 0x0,
        .size = 0x0,
    },
    ...
    {
        .num = USER_MEM_ID,
        .start_phys = 0x0,
        .start_virt = 0x0,
        .size = 0x0,
    },
}
```

步骤3. 在设备树文件中添加保留内存的节点

```
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
scp: scp@1cb00000{
    ...
    scp_mem_tbl = <0 0x0>,
    <1 0xca700>, /* vow */
    <2 0x100000>, /* sensor main*/
    ...,
    id, size>; /*user_mem*/
};
```

步骤4. 在 Makefile 中增加 SCP 共享内存的大小

```
{lk2_repo}/platform/mediatek/mt6897/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x          //在此变量中添加增量
```

6.1.2 扩展共享内存

以传感器为例，增加 1.5MB 的内存

```
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
scp: scp@1cb00000{
...
scp_mem_tbl = <0 0x0>,
<1 0xca700>, /* vow */
<2 0x280000>, /* sensor main original value is 0x100000 */
...
};
```

```
{lk2_repo}/platform/mediatek/mt6897/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x680000    // original = 0x500000
```

6.2 通过 ID 获取保留内存

由于 AP CPU 使用虚拟地址，而 SCP 使用物理地址，因此需要提供这两种类型的地址。以下 API 用于通过给定的 ID 获取虚拟/物理地址和大小，这些 ID 在章节 6.1 中声明。

- API


```
phys_addr_t scp_get_reserve_mem_phys(scp_reserve_mem_id_t id)
phys_addr_t scp_get_reserve_mem_virt(scp_reserve_mem_id_t id)
phys_addr_t scp_get_reserve_mem_size(scp_reserve_mem_id_t id)
```

- 头文件路径

```
master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp/rvMt8676/scp_
helper.h
```

- 返回值

- 保留内存的起始地址，或
- 0x0：表示没有映射

在获取物理地址后，开发人员必须通过 IPI 将其传递给 SCP，如章节 5.2 所述。

6.3 在 SCP 端获取保留的 DRAM 区域

为了保持安全状态并避免恶意的 IPI 消息，SCP_DRAM_REGION 被引入以直接提供 SCP 端保留的 DRAM 区域信息。

以下 API 用于通过给定的 ID 获取物理地址和大小，这些 ID 在章节 6.1 中声明。

- API

```
bool scp_get_reserve_mem_by_id(uint32_t id, void **ap_addr, size_t *size)
```

- 头文件路径

```
master/src/tinysys/mt8676/scp/drivers/common/dram_region_mgmt/scp_dram_region.h
```

- 返回值

- True: *ap_addr 和 size 的值被更新为给定 ID 的起始地址和大小。
- False: *ap_addr 和 size 的值未被更新。

对于可能在不同项目中通用的代码，CFG_SCP_DRAM_REGION_MANAGE 可以帮助保持兼容性。例如：

```
#ifdef CFG_SCP_DRAM_REGION_MANAGE
#include "scp_dram_region.h"
#define USER_MEM_ID 11 //id number from dts
#endif

void init_memory(...) {
    ...
    #ifdef CFG_SCP_DRAM_REGION_MANAGE
    void *addr;
    size_t size;

    if (scp_get_reserve_mem_by_id(USER_MEM_ID, &addr, &size))
        set_dram_addr(addr, size);
    else
        ...
    #else
    // 从 IPI 消息中获取地址和大小
    set_dram_addr(msg.addr, msg.size);
    #endif
    ...
}
```

6.4 将 DRAM 地址从 AP 视图重映射到 SCP 视图

SCP 是一个 32 位系统，最多只能访问 4G (0xffffffff) 地址。如果 SCP 应用程序需要访问高于 4G 的地址，我们需要使用以下 API 将地址重映射到 SCP 内存窗口。以下是 AP 和 SCP 之间的映射表。

表 6-2. 默认重映射规则表

名称	SCP/DMA		大小	AP 端	
External Memory	0x1000_0000	0x1FFF_FFFF	256MB	0x6000_0000	0x6FFF_FFFF
External Memory	0x2000_0000	0x2FFF_FFFF	256MB	0x7000_0000	0x7FFF_FFFF
External Memory	0x5000_0000	0x5FFF_FFFF	256MB	0x0000_0000	0x0FFF_FFFF
External Memory	0x6000_0000	0x6FFF_FFFF	256MB	0x1000_0000	0x1FFF_FFFF
External Memory	0x9000_0000	0x9FFF_FFFF	256MB	0x8000_0000	0x8FFF_FFFF
External Memory	0xA000_0000	0xAFFF_FFFF	256MB	0x9000_0000	0x9FFF_FFFF
External Memory	0xD000_0000	0xDFFF_FFFF	256MB	0x2000_0000	0x2FFF_FFFF
External Memory	0xE000_0000	0xEFFF_FFFF	256MB	0x3000_0000	0x3FFF_FFFF
External Memory	0xF000_0000	0xFFFF_FFFF	256MB	0x5000_0000	0x5FFF_FFFF

- API
 - uint32_t ap_to_scp(uint32_t ap_addr);
 - uint32_t scp_to_ap(uint32_t scp_addr);

- 头文件路径

master/src/tinysys/mt8676/scp/drivers/common/dma/inc/dma_api.h

- 返回值

- 映射地址, 或
- 0x0: 表示没有映射

6.5 请求系统总线和 DRAM

由于在没有数据传输或系统挂起时, 系统总线和 DRAM 会进入睡眠模式, 因此必须调用以下 API 以确保 DRAM 可以被访问。

- 用于管理任务的 API void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id): before DRAM access
void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id): after DRAM access

注: 当 26M 时钟被关闭时, 唤醒 DRAM 需要 5 毫秒

- 用于管理中断服务程序的 API

```
void dvfs_enable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)
void dvfs_disable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)
```

- 头文件路径

master/src/tinysys/mt8676/common/drivers/dma/v3/inc/dma.h
master/src/tinysys/mt8676/scp/drivers/RV55_A/Mt8676/dvfs/inc/dvfs.h

7 驱动程序指南

7.1 驱动程序初始化

由于在驱动程序初始化时未启用多线程，开发人员必须

- 在 `platform_init()` 中初始化驱动程序
- 避免在驱动程序初始化函数中使用阻塞函数，因为这会导致永远阻塞：
 - `vTaskDelay`：用于在 FreeRTOS 任务中延迟执行一段时间
 - HW semaphore：硬件信号量，用于在任务之间进行同步和通信
 - Busy loop, e.g., polling registers：忙等待，例如轮询寄存器，用于等待某些条件的发生

7.2 添加新驱动程序

要添加新驱动程序，请按照以下步骤操作：

1. 在适当的目录中创建新的驱动程序文件 (选择如下的目录)

- 路径

```
master/src/tinysys/mt8676/common /* 通用的 tinysys 驱动程序 */
master/src/tinysys/mt8676/scp/drivers/common /* scp 通用的驱动程序 */
master/src/tinysys/mt8676/scp/drivers/RV55_A/Mt8676/drivers /* 平台相关的驱动 */
```

2. 添加新的编译选项

- 路径

```
master/src/tinysys/mt8676/scp/project/Mt8676/platform/platform.mk
```

- 示例：DMA 驱动程序

```
CFG_DMA_SUPPORT = yes
...
ifeq ($(CFG_DMA_SUPPORT), yes)
    INCLUDES += $(COMMON_DIR)/drivers/dma/v3/inc
    INCLUDES += $(SCP_DRIVERS_DIR)/common/dma/inc
    INCLUDES += $(DRIVERS_PLATFORM_DIR)/dma
    C_FILES += $(COMMON_DIR)/drivers/dma/v3/dma.c
    C_FILES += $(SCP_DRIVERS_DIR)/common/dma/dma_api.c
endif
```

7.3 中断

Mt8676 SCP 支持 15 个优先级的中断，优先级数字越低，优先级越高。如果同时发生多个 IRQ，CPU 将优先处理优先级最高的 IRQ。绝对不要将中断优先级设置为 2 以上。0 级用于“看门狗”或“系统故障”，1 级用于“睡眠控制”中断。我们将在本章描述如何在 SCP 中使用中断。

- 路径

master/src/tinysys/mt8676/common/drivers/irq/v3/inc/irq.h

7.3.1 IRQ 注册

在 IRQ 被处理之前，驱动程序开发人员必须在 IRQ ID 和相应的处理程序之间建立关联。IRQ 根据 INTC_IRQ 的结构定义，驱动程序开发人员使用以下 API 注册处理程序和 IRQ ID。

- 架构

```
struct INTC_IRQ
```

```
{
    uint8_t id;
    uint8_t group;
    uint8_t pol;
}
```

id: IRQ 编号

group: 优先级组，从 INTC_GRP_0（最高优先级）到 INTC_GRP_14（最低优先级）

pol: 触发极性，使用 INTC_POL_HIGH 或 INTC_POL_LOW

- API

```
int intc_irq_request(struct INTC_IRQ *irq, irq_handler_t handler, void *userdata)
```

Description

Request an irq and register the handler, and default enable the irq handler.

Parameters

irq: the irq structure which is declared at intc.h

handler: irq handler

userdata: it will deliver to irq handler as a parameter

Return values

0: success

-1: request fail

- 示例

```
struct INTC_IRQ INTC_IRQ_SYSTICK = {0, INTC_GRP_8, INTC_POL_HIGH};
```

```
#include "irq.h"
```

```
int ret;
```

```
ret = intc_irq_request(&INTC_IRQ_SYSTICK, test_ist, NULL);
```

```
if (ret != 0)
```

```
PRINTF_E("Register irq failed\n");
```

7.3.2 使能中断

在 IRQ 处理程序注册后，SCP 已准备好服务。下一步是使用以下 API 启用特定的 IRQ。

- API

```
int intc_irq_enable(struct INTC_IRQ *irq)
```

Description

启用指定的 IRQ 服务。

Parameters

irq: 在 intc.h 中声明的 IRQ 结构体

Return values

0: 成功

-1: 请求失败

- 示例:

```
#include "irq.h"

int ret;
ret = intc_irq_enable(&INTC_IRQ_SYSTICK);
if (ret != 0)
    PRINTF_E("enable irq failed\n");
```

7.3.3 禁用 IRQ

相反，当 IRQ 暂时不使用时，驱动程序开发人员必须调用以下 API 使 SCP 停止服务这些 IRQ。

- API

```
int intc_irq_disable(struct INTC_IRQ *irq)
```

Description

禁用指定的 IRQ 服务。

Parameters

irq: 在 intc.h 中声明的 IRQ 结构体

Return values

0: 成功

-1: 请求失败

- 示例

```
#include "irq.h"

int ret;
ret = intc_irq_disable(&INTC_IRQ_SYSTICK);
```

```
if (ret != 0)
    PRINTF_E("disable irq failed\n");
```

7.3.4 注册唤醒源

当 SCP 处于睡眠状态时，除非设置为唤醒源，否则 IRQ 将不会被处理。以下 API 用于将 IRQ 注册为唤醒源。

- API

```
int intc_irq_wakeup_set(struct INTC_IRQ *irq, unsigned int wake_src)
```

Description

为指定的 IRQ 设置唤醒源。

Parameters

irq: 在 intc.h 中声明的 IRQ 结构体

wake_src: 1 表示唤醒源, 0 表示非唤醒源

Return values

0: 成功

-1: 请求失败

- 示例

```
#include "irq.h"

int ret;
ret = intc_irq_wakeup_set(&INTC_IRQ_SYSTICK, 1);
if (ret != 0)
    PRINTF_E("irq wakeup source setup failed\n");
```

注意:

1. ISR 执行时间必须尽可能短，堆栈深度尽可能小。
2. 绝对不要在 ISR 中使用阻塞 API，例如：
 - 硬件信号量 API
 - 等待 IPI
 - 任何带有轮询外部设备的 API
3. 绝对不要在 ISR 中使用没有 FromISR 后缀的 FreeRTOS API。
 - 这些是重写版本，具有无阻塞 API 和快速实现。
4. 如果在 ISR 中唤醒了高优先级任务，必须使用 portYIELD_FROM_ISR()。

7.4 锁

SCP 提供了用于双核同步的自旋锁机制。使用锁需要打开如下配置项：

```
CFG_ATOMIC_PLAT_SUPPORT = yes
```

- 路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/platform.mk
```

- API

```
spinlock_t SYNC_SECTION lock;
```

```
void spinlock_lock(spinlock_t * lock)
```

```
void spinlock_unlock(spinlock_t * lock)
```

```
description
```

获取/释放自旋锁

```
parameters
```

lock: SYNC_SECTION 定义的变量

```
Return values
```

无。

- 头文件路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/inc/mtk_atomic.h
```

注意：

1. 绝对不要保持自旋锁超过 1 毫秒，因为在自旋锁和解锁期间会禁用抢占。
2. 对于不跨越双核的场景，不需要使用自旋锁。

7.5 DMA

直接内存访问（DMA）是一种支持在指定源/目标之间复制数据而不涉及 CPU 的硬件。SCP DMA 支持以下功能：

1. 突发 AXI 模式以加速内存传输
2. 8 个通道，即引擎最多可以同时操作 8 个事务。

- API

```
DMA_RESULT scp_dma_transaction(uint32_t dst_addr, uint32_t src_addr, uint32_t len, int8_t  
scp_dma_id, int32_t ch)
```

```
DMA_RESULT scp_dma_transaction_dram(uint32_t dst_addr, uint32_t src_addr, uint32_t len,  
int8_t scp_dma_id, int32_t ch)
```

```
description
```

通过指定的 DMA 通道从 src_addr 复制数据到 dst_addr。

```
parameters
```

```

dst_addr: 目标地址
src_addr: 源地址
len: 要复制的字节长度
scp_dma_id: DMA ID
ch: channel 通道 ID
Return values
DMA_RESULT_DONE (=0) 表示成功开始
DMA_RESULT_NO_FREE_CH (=-1) 表示 DMA 硬件忙

```

• 头文件路径

```

master/src/tinysys/mt8676/common/drivers/dma/v3/inc/dma.h
master/src/tinysys/mt8676/scp/drivers/common/dma/dma_api.h
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/mt_dma.h

```

• scp_dma_id

- scp_dma_id 是在 mt_dma.h 中的 DMA 通道的标识。建议使用不同的 dma_id，这样会更容易调试出通道满的问题。

• 限制

- 每个事务的最大数据大小为 262140 字节
- 使用 4 字节对齐的地址以获得最佳性能

• 示例:

```

ret = scp_dma_transaction(dst_buf + dst_w_pos, src_buf + src_r_pos, src_len,
LOGGER_DMA_ID, NO_RESERVED);
if (ret != DMA_RESULT_DONE) {
    PRINTF_E("log dma trans fail%u\n", ret);
    return 0;
}

```

7.6 硬件信号量

硬件信号量是一种特殊的硬件，它在 Linux 驱动程序和 FreeRTOS 之间提供类似互斥锁的流控制。在 SCP 中有 16 组硬件信号量。以下 API 使硬件信号量易于使用，它们在 SCP 和 Linux 驱动程序中都可以工作。只需包含正确的头文件并确保标志相同。

• API

```

int semaphore_get(unsigned int flags)
int semaphore_release(unsigned int flags);
    description
        在 AP 和 SCP 之间使用的信号量。
    parameters

```

flag: 0 ~ 15, 对应 SCP 中的 16 组信号量

Return values

- 0: 获取信号量失败
- 1: 获取信号量成功

- 头文件路径

- master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp/rv/scp_helper.h
 - master/src/tinysys/mt8676/common/drivers/sem/v1/inc/sem.h

- 返回值

- 1: 成功获取信号量
 - 0: 获取信号量失败

- 示例:

```
int get_semaphore;  
  
while(1) {  
    get_semaphore = semaphore_get(4)  
    if (get_semaphore)  
        break;  
}
```

7.7 GPIO 和 EINT

SCP 还提供 GPIO 和外部中断 (EINT)，以便外部组件（如陀螺仪传感器）可以向 SCP 发送事件。

- GPIOs
 - Function set to: TP_GPIO?_AO, 支持 TP_GPIO0_AO~TP_GPIO15_AO
- EINTs
 - Function set to Aux Func.0(GPIO), 支持 EINT0~15

表 7-1. EINT 和 GPIO 的映射引脚名称

Ball name	GPIO Reset Default Mode	EINT	Aux Func.0	Aux Func.4	Aux Func.6
EINT0	0	EINT0	B:GPIO0		B0:TP_GPIO0_AO
EINT1	0	EINT1	B:GPIO1		B0:TP_GPIO1_AO
EINT2	0	EINT2	B:GPIO2		B0:TP_GPIO2_AO
EINT3	0	EINT3	B:GPIO3		B0:TP_GPIO3_AO
EINT4	0	EINT4	B:GPIO4		B0:TP_GPIO4_AO
EINT5	0	EINT5	B:GPIO5		B0:TP_GPIO5_AO
EINT6	0	EINT6	B:GPIO6		B0:TP_GPIO6_AO
EINT7	0	EINT7	B:GPIO7		B0:TP_GPIO7_AO
EINT8	0	EINT8	B:GPIO8	B0:TP_GPIO8_AO	
EINT9	0	EINT9	B:GPIO9	B0:TP_GPIO9_AO	
EINT10	0	EINT10	B:GPIO10		B0:TP_GPIO10_AO
EINT11	0	EINT11	B:GPIO11		B0:TP_GPIO11_AO
EINT12	0	EINT12	B:GPIO12		B0:TP_GPIO12_AO
EINT13	0	EINT13	B:GPIO13		B0:TP_GPIO13_AO
EINT14	0	EINT14	B:GPIO14		B0:TP_GPIO14_AO
EINT15	0	EINT15	B:GPIO15		B0:TP_GPIO15_AO

7.7.1 GPIO 使用

GPIO 功能必须首先设置为 TP_GPIO?_AO。请参考 GPIO 引脚复用设置文档以获取详细信息。表 7-2 是 GPIO 控制寄存器表

表 7-2. GPIO 控制寄存器表

Register Offset	Name	Description	Access	Enumeration
0x25000	GPIO_DIR	GPIO direction	RW	0: Input 1: Output
0x25004	GPIO_OUT	GPIO output	RW	GPIO output [1:0]
0x25008	GPIO_IN	GPIO input	RO	GPIO input[1:0]
0x2500C	GPIO_PULL_EN	GPIO pull enable	RW	0: Disable 1: Enable
0x25010	GPIO_PULL_CTRL	GPIO pull control	RW	0: Pull down 1: Pull up

控制示例：

- 拉高 GPIO 1
 - Set GPIO_DIR, 0x60525000[1] = 1
 - Set GPIO_OUT, 0x60525004[1] = 1
- 读取 GPIO 0
 - Set GPIO_DIR, 0x60525000[0] = 0
 - Read GPIO_IN, 0x60525008[0]
- 内部上拉 GPIO 3
 - Set GPIO_PULL_EN, 0x6052500c[3] = 1
 - Set GPIO_PULL_CTRL, 0x60525010[3] = 1

7.7.2 EINT 使用

- 路径

tinysys/mt8676/common/drivers/eint/v02/src

- API: 注册 EINT 回调

```
void mt_eint_registration(unsigned int eint_num, unsigned int sens, unsigned int pol,
                          void (EINT_FUNC_PTR) (int),
                          unsigned int unmask, unsigned int
                          is_auto_unmask)
    description
```

注册 EINT 中断处理程序。

parameters

eint num: EINT 编号

sens: 触发类型 (LEVEL_SENSITIVE 或 EDGE_SENSITIVE)

pol: 触发极性 (HIGH_LEVEL_TRIGGER 或 LOW_LEVEL_TRIGGER)

EINT_FUNC_PTR: 中断服务程序 (ISR) 回调函数

Unmask: 注册后是否启用此 EINT 触发

Is_auto_unmask: 完成 EINT 服务例程后是否自动重新启用 EINT 触发

Return values

无

- 示例

```
mt_eint_registration(eint_num, LEVEL_SENSITIVE, HIGH_LEVEL_TRIGGER, xxx_Isr,
EINT_INT_UNMASK, EINT_INT_AUTO_UNMASK_OFF);
```

- void mt_eint_dis_hw_debounce(unsigned int eint_num): 禁用硬件去抖动功能
- void mt_eint_soft_set(unsigned int eint_num) : 使用软件触发来清除指定的外部中断

8 调试方法

8.1 PRINTF_* 使用

请不要在 SCP 软件中使用 printf，因为它可能会链接到 C 库并引起问题。相反，我们使用 PRINTF_*，如下表所示。

开发人员必须在使用 PRINTF_* 之前包含头文件 <mt_printf.h>。PRINTF_* 的日志级别和使用场景如下表所示。

表 8-1. PRINTF 使用场景

PRINTF_*	级别	使用场景	
PRINTF_E	<0>	错误信息	
PRINTF_W	<1>	警告信息	使用者级别镜像
PRINTF_I	<2>	提示信息	
PRINTF_D	<3>	调试级别信息	工程师级别镜像

8.2 串口

- 输出引脚

SCP 有两个个专用的 UART。请确保 PC 的 UART 端口连接到引脚名称 URXD1 和 UTXD1，如下表 8-2 所示，并设置软件编译选项。

表 8-2. UART 引脚名称

引脚名称	功能
URXD1	SCP UART RX
UTXD1	SCP UART TX

- 软件编译选项

- 路径：Configure flags

master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/platform.mk

CFG_UART_SUPPORT = yes /* 启用 UART, 默认 No */

CFG_MTK_SCPUART_SUPPORT = yes /* 使用 SCP UART, 默认 Yes */

- UART 终端设置

- 波特率：921600

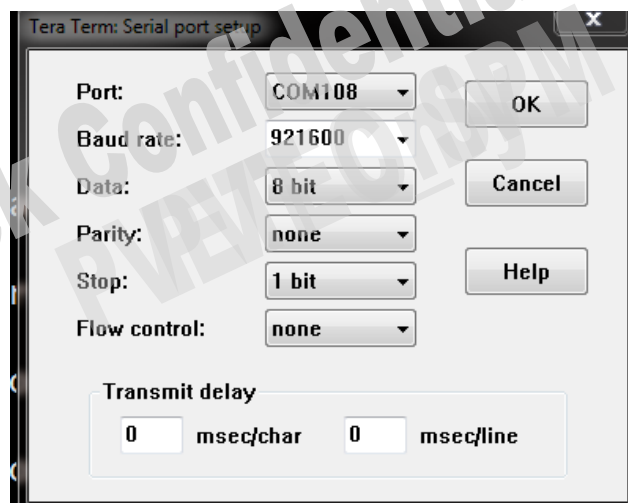


图 8-1. 串口工具设置

8.3 异常日志分析

当异常发生时，SCP 会自动打印异常日志。开发人员可以通过 UART 或 mobile 日志获取这些日志信息。

- FAULT FETCH, FAULT LOAD 和 FAULT STORE：由于访问了 MPU 保护地址导致的错误。
 - FAULT FETCH：PC 跳转到错误地址
 - 系统寄存器 mepc 将显示错误的目标地址
 - FAULT LOAD, FAULT STORE：访问了受保护的地址
 - 系统寄存器 mepc 将显示错误的 PC，mtval 将显示错误的访问地址。

```
[70.168] (0) exception: CAUSE_FAULT_LOAD
[70.168] (0)      exception pc: 0x00018b2c
[70.168] (0)      fault load address: 0xfffff1110
[70.168] (0) Regs dump
[70.168] (0) x0: 0xca0801e4 ra: 0x00018b24
[70.168] (0) sp: 0x00036ec0 gp: 0x00000000
...
[70.168] (0) t3: 0x00000000 t4: 0x00000011
[70.168] (0) t5: 0x00000000 t6: 0x00000000
[70.168] (0) pc: 0x00001454 mstatus: 0x05015800
[70.168] (0) mepc: 0x00018b2c
[70.168] (0) mcause: 0x00000005
[70.168] (0) mtval: 0xfffff1110
[70.168] (0) T Buffer (10)
[70.168] (0) 00 0x0000fdca:0x0002252c
[70.168] (0) 01 0x0002252c:0x0000fdce
...
[70.169] (0) 28 0x000108c0:0x0002368e
[70.169] (0) 29 0x00023698:0x000108c4
[70.169] (0) 30 0x000108ca:0x00024518
[70.169] (0) 31 0x00024524:0x00018b24
```

[70.169] (0) Code: 7545 85aa 0513 1105 <4108> 8593 1145 c188 9205

MISALIGNED FETCH, MISALIGNED LOAD, and MISALIGNED STORE: 通常发生在整数指针（4 字节对齐）访问 1 或 2 字节对齐的地址时。

- MISALIGNED FETCH: 跳转到未对齐的地址
 - 系统寄存器: mepc 将显示错误的地址
- MISALIGNED LOAD, MISALIGNED STORE: 访问未对齐的地址
 - 系统寄存器 mepc 将显示错误的 PC, mtval 将显示访问地址

由于指令无法解码导致的错误。示例：

```
[2.643] (0) exception: CAUSE_ILLEGAL_INSTRUCTION
[2.643] (0) Regs dump
[2.644] (0) x0: 0xa5a5a5a5 ra: 0xa5a5a5a5
[2.644] (0) sp: 0xa5a5a5a5 gp: 0xa5a5a5a5
...
[2.651] (0) t3: 0x00000000 t4: 0x00000000
[2.652] (0) t5: 0x00000000 t6: 0x00000000
[2.652] (0) pc: 0x00001454 mstatus: 0x05015880
[2.653] (0) mepc: 0x00201720
[2.654] (0) mcause: 0x00000005
[2.654] (0) mtval: 0x2ff13748
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
[2.664] (0) Code: d194 5133 4521 873f <3748> 2ff1 a5a5 a5a5 a5a5
```

8.3.1 Trace Buffer

Trace buffer 日志可以帮助我们分析 CPU 执行的函数调用记录。日志显示了函数调用地址和目标地址。共有 32 个条目，可以从异常日志的底部获取。

示例：

该地址可以使用诸如 lldb 或 addr2line 等工具进行转换

```
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
```

在执行 coredump_cmd.sh 脚本处理 SCP_COREDUMP 文件后，会生成一个可读的文本文件 analysis.txt。该文件列出了函数调用地址、函数名称和源文件路径的组合。

```
C0 0: 0x0000371a: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/../../../../adsp_ipi_queue.c:522
C0 1: 0x00008fd6: xQueueSemaphoreTake at alps-dev-r0_mp3/vendor/../../../../queue.c:1599 (discriminator 6)
C0 2: 0x00008fdc: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/../../../../adsp_ipi_queue.c:500
C0 3: 0x0000371e: scp_sleep_ctrl at alps-dev-r0_mp3/vendor/../../../../sleep.c:377
```

8.4 核心转储

如果 AP 检测到 SCP 没有响应 IPI 或发生 SCP 看门狗事件，核心转储流程将自动启动。核心转储是 SCP 内存和处理寄存器（如程序计数器、堆栈指针、返回地址等）的快照，并尽可能多地保存系统信息（如系统寄存器、缓存内容）。这些信息使得在故障发生前恢复系统状态成为可能。

默认情况下，LLDB 被用于支持核心转储调试。有关 LLDB 的详细信息，请参考 LLDB 官网：<https://lldb.llvm.org/>。

- LLDB 路径
 - LLDB 可以在 `prebuilts` 文件夹中找到，路径为 `prebuilts/devtool/clang/md32rv/linux-x86/lldb_v3`。
- 获取核心转储
 - SCP 核心转储将命名为 `SYS_SCP_DUMP`，位于 SCP EE DB（例如，`ex.db.00.EE.dbg`）中。如果未找到，请检查 EE DB，确认 `__exp_main.txt` 中的异常类型应为 `scp`。
- 开始调试
- 输入以下命令，初始日志如下所示

```
$ prebuilts/devtool/clang/md32rv/linux-x86/lldb_v3/coredump_cmd.sh mt6897 tinysys-scp-RV55_A.elf
SCP_COREDUMP 0
```

```
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/freertos.py
(lldb) freertos

FreeRTOS Awareness is working ...

CPU 0 -- OS Tick: 70125
CPU 1 -- OS Tick: 0
OS is in normal state (neither in Critical Section nor in HW ISR)
TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
Stack (unit in word) size=1024, current used=124, max used=197
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000)
at exception.c:1055
frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
...
Flight Recorder is working ...
No Flight Recorder Support
(lldb)
```

注意：

LLDB 版本必须与 SCP 核心兼容

RV55 : lldb_v3

RV33 : lldb

如果 LLDB 工具意外启动：

1. 请确认 `prebuilts/devtool/clang/md32rv/linux-x86/lldb_v3` 文件结构未被修改。
2. 请检查相关日志文件 `debug_prosim.log` 和 `debug_ocd.log`。如果未找到 `libprofile.so.x.x.x` 库，请在 `coredump_cmd.sh` 中的 PROSIM 启动命令之前添加以下命令。

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$MRV_PDK_PROSIM_HOME  
  
#add above this command  
$PROSIM_EXE $PROSIM_OPT
```

3. 使用 `coredump_cmd.sh` 时会启动三个工具（LLDB、Openocd、PROSIM），工具之间通过 TCP/IP 协议进行通信。如果远程使用 LLDB 相关工具，请确认 TCP/IP 连接端口未被防火墙阻止。

8.4.1 LLDB 基本命令

对于高级开发人员或熟悉 GDB 的开发人员，请参考以 [command map](#) 以了解详细信息。

- 通过命令 "bt" 来获取函数调用的回溯信息

```
(lldb) bt
* thread #1, name = 'Hart0', stop reason = instruction step into
  * frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000)
    at exception.c:1055
    frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
      prevVal=238376, newVal=<unavailable>) at atomic.c:181
    frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
      osSetCurrentTask(task=0x0003a328) at seos.c:153
    frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
      seos.c:212
    frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
      timer.c:145
    frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
      jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
      oneShot=<unavailable>) at timer.c:179
    frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
      cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
    frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
```

- 通过命令 `freertos` 来获取操作系统任务的信息

```
(lldb) freertos
Find 18 tasks
CPU 0 -- OS Tick: 70125
CPU 1 -- OS Tick: 0
Coredump cpu: 0
OS is in normal state (neither in Critical Section nor in HW ISR)

TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
Stack (unit in word) size=1024, current used=124, max used=197
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000)
at exception.c:1055
    frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
    frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
osSetCurrentTask(task=0x0003a328) at seos.c:153
    frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
seos.c:212
    frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
timer.c:145
    frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
oneShot=<unavailable>) at timer.c:179
    frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
    frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
...

TASK-17: CHRE@cpu0 ( TCB: 0x0003f910, State: Suspended, TCB Number: 4, Priority: 4)
Stack (unit in word) size=1024, current used=132, max used=364
* frame #0: 0x00013bc4 tinysys-scp-RV55_A.elf`vTaskExitCritical at tasks_smp.c:4287
    frame #1: 0x00007aee tinysys-scp-RV55_A.elf`evtQueueDequeue(q=<unavailable>,
evtTypeP=<unavailable>, evtDataP=<unavailable>, evtFreeDataP=<unavailable>,
sleepIfNone=<unavailable>) at eventQ.c:171
    frame #2: 0x0000cbb8 tinysys-scp-RV55_A.elf`osMainDequeueLoop at seos.c:1052
    frame #3: 0x0000cdfc tinysys-scp-RV55_A.elf`osMain at seos.c:1093
```

- 通过命令 `p $(variable name)` 来打印变量和地址

```
(lldb) p mTask
(alsPsTask) $0 = {
    id = 259
    handle = ([0] = 16973827, [1] = 16973828, [2] = 16973829)
    prevRtcTime = 0
    mDataSlab = 0x0003c340
    dataEvt = 0x00000000
    alsLastSample = 0
    psLastSample = 1
    bcRecv = {
        [0] = {
            list = {
                prev = 0x00048290
                next = 0x002109e0
            }
            sensor_type = '\f'
            receive_event = 0x00206a34 (tinysys-scp-RV33_A.elf`alsPsReceiveEvent at alsps.c:908)
        }
    }
}
```

- 通过命令 `x/FMT address` 来转储内存

```
(lldb) x/32xw 0x1ef08
0x0001ef08: 0x003a9593 0x093d3533 0x10063667 0x00bb0ab3
0x0001ef18: 0x01340633 0x016ab5b3 0x00b60c33 0x008c0463
0x0001ef28: 0x008c35b3 0x01248633 0x0433955e 0x35b300b6
0x0001ef38: 0x36330096 0x952e00c4 0x00c50bb3 0xd5334d32
0x0001ef48: 0xc11d094b 0x001ad593 0x001af513 0xfe1c05b3
0x0001ef58: 0x001c5c13 0xfe140c33 0x84338005 0xdb93fe1b
0x0001ef68: 0xeab3001b 0x088500a5 0x652144a2 0x09f4a4b3
0x0001ef78: 0xcf63157d 0x053700a8 0x24237fff 0x8d451001
```

- 通过命令 `register read $REG` 来读取 CPU 寄存器的值

```
(lldb) register read pc
pc = 0x00000038 tinysys-scp-RV33_A.elf`__divtf3 + 54 at divtf3.c
(lldb) register read
general:
    x0 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x1 = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
    ...
    x27 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x28 = 0x0000006d tinysys-scp-RV33_A.elf`__divtf3 + 107 at divtf3.c:30
(lldb) register read mepc
mepc = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
```

8.5 性能评估和运行记录

PBFR 是一种性能分析工具，可以监控 CPU 使用情况。它可以分为两个部分：性能评估和运行记录。

性能评估包含每个任务的负载、缓存未命中、停顿和整个系统负载的信息。运行记录可以在一段时间内记录 CPU trace。

8.5.1 性能评估

这里将介绍如何使用性能预算来监控任务负载。

- 启用性能评估：
 - 设置配置和定义以启用性能评估。
 - 路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/inc/FreeRTOSConfig.h
master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/platform.mk
```

表 8-3. 编译器选项和定义

#	编译器选项和定义	描述
1	CFG_PBFR_SUPPORT	PBFR 主功能控制选项 Enable: Yes Disable: No
2	#define PBFR_SUPPORT_POLLING	支持通过轮询保存任务最大负载
3	#define PBFR_SUPPORT_POLLING_MS 10	每 10ms 轮询一次任务活动
4	#define PBFR_SUPPORT_CACHE_COUNT	支持缓存未命中率
5	#define PBFR_SUPPORT_IOSTALL	支持 IO 停顿率

- 使用方法 1：
 - 每 10 秒报告一次性能预算
 - CFG_MONITORTASK_ALWAYS_RUN = yes
- 使用方法 2：
 - 命令：（开始/报告/结束）\$ echo 666 0 1 > sys/class/misc/scp/scpctl
 - 第一次输入命令开始记录性能预算信息，在一段时间后，第二次输入命令报告负载信息，第三次输入命令停止记录。
 - 对于一些定制化的请求，可以在特定位置添加这三个 API 以监控部分任务。
 - pbfr_start_loadinfo(0)：开始记录负载信息

- pbfr_report_loadinfo(0): 报告负载信息
- pbfr_stop_loadinfo(0): 停止记录负载信息

• 日志解释:

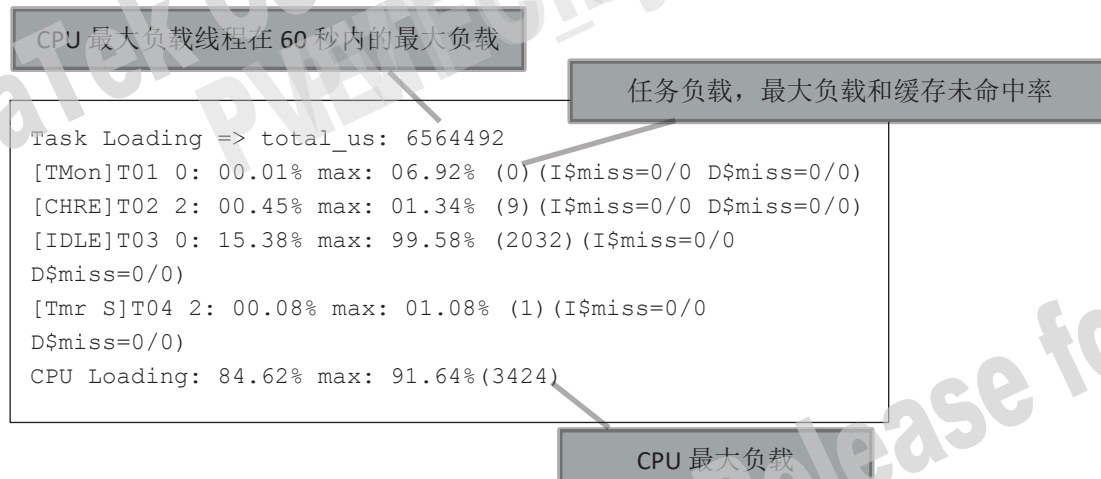


图 8-2. 性能评估日志示例

8.5.2 运行记录

运行记录是一种调试工具，可以在系统崩溃时记录最后的事件。这里将介绍如何使用飞行记录来监控 CPU 跟踪。

- 启用运行记录：
 - 设置配置并打开定义以获取运行记录。platform.mk 中的配置用于启用运行记录，而 FreeRTOSConfig.h 中的定义表示可以记录的事件。

• 路径

master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/platform.mk

表 8-4. 编译器选项

#	编译器选项	解释
	CFG_PBFR_SUPPORT	PBFR 主功能控制选项 Enable: Yes Disable: No

• 路径

master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/inc/FreeRTOSConfig.h

表 8-5. Flight Record 定义

#	飞行记录名称	解释
1	#define PBFR_SUPPORT_FLIGHT_REC	支持飞行记录
2	#define PBFR_MAX_REC_EVENTS 32	支持 32 个跟踪事件（最大：255）
3	#define PBFR_SUPPORT_REC_TASK	支持任务跟踪事件
4	#define PBFR_SUPPORT_REC_QUEUE	支持队列跟踪事件
5	#define PBFR_SUPPORT_REC_SWTIMER	支持软件定时器跟踪事件
6	#define PBFR_SUPPORT_REC_INT	支持关键部分跟踪事件
7	#define configTRACE_INT_MASK_TIME_BOUD_NS	仅在持续时间超过时间界限时记录关键部分跟踪事件
8	#define PBFR_SUPPORT_REC_ISR	支持中断服务例程跟踪事件
9	#define PBFR_SUPPORT_REC_OSTICK	支持操作系统时钟跟踪事件

- 如果构建因内存大小不足而失败，请在 Setting.ini 中更改设置。

日志：SCP: {XXX}(size_A>size_B) is out of memory limitation

将 XXX 的大小（size_B）调大至大于 size_APath

master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/Setting.ini

- 使用方法:

- 将 tinysys-scp-RV55_A.elf 和 SCP_coredump 复制到分析文件夹中。
 - 分析文件夹: prebuilts/devtool/clang/md32rv/linux-x86/lldb_v3
- 使用核心转储分析工具解析数据以获取 SCP_debug.txt 和 flrec.json。
- 如果成功，将显示以下日志。

```
(lldb)command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/flrec.py
(lldb) flrec
Flight Recorder is working ...
Flight Record output done.
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/fflrec.py
(lldb) fflrec
Flight Recorder is working ...
('xTickCount:', 82801L)
len=1000 status=00000003 RP=123 WP=123 last=82809147582
virtual ns = 82320378580
Flight Record output done.
(lldb)
```

在 chrome://tracing/ 中加载 flrec.json 以获取 GUI 结果。

- 日志解释:

- 控制台模式:

- 控制台模式将在 SCP_debug.txt 中显示, 它显示系统崩溃前记录的事件, 并按类型分类显示事件。

Record[X] 表示这是倒数第 X 个事件。



图 8-3. 飞行记录控制台模式示例

- GUI 模式：
 - 加载步骤
 1. GUI 网站: chrome://tracing/
 2. 加载 json 文件: flrec.json (分析文件夹中创建)
 - 使用说明：
 - 使用键盘 W/S 缩放，A/D 左右移动，或切换按钮以更改鼠标模式。



图 8-4. 操作方法

– 总览:

- GUI 显示最后的事件流，记录六种类型的事件：任务、队列事件、软件定时器、关键部分、ISR 事件和操



图 8-5. 运行记录的 GUI 显示模式

- 底部的消息显示事件的开始时间和持续时间。



图 8-6. 详细信息显示

– 任务

- 任务事件显示任务的唤醒时间和执行时间。



图 8-7. 任务被唤醒的事件

– 队列

- 队列事件显示在执行事件的任务下方，点击事件可以在信息栏中查看队列类型。

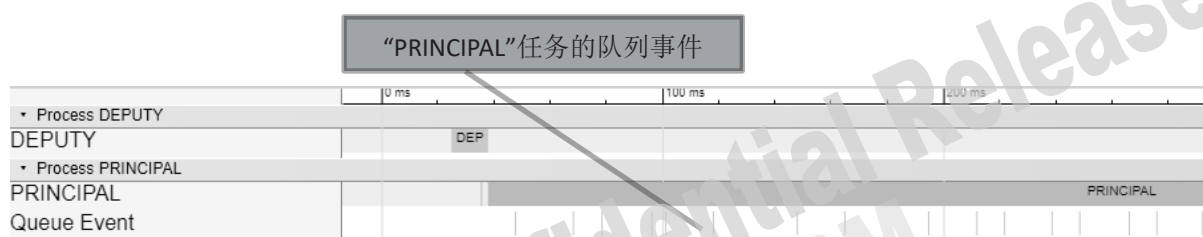


图 8-8. 队列事件的示例



图 8-9. 队列事件的详细信息

– 软件定时器

- 软件定时器显示回调函数的执行时间。

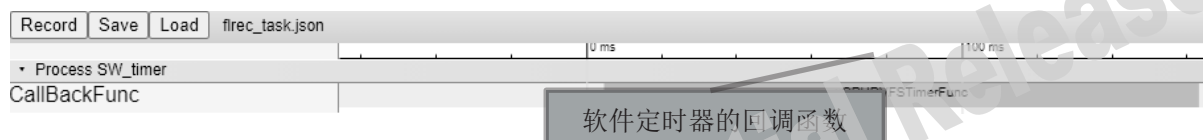
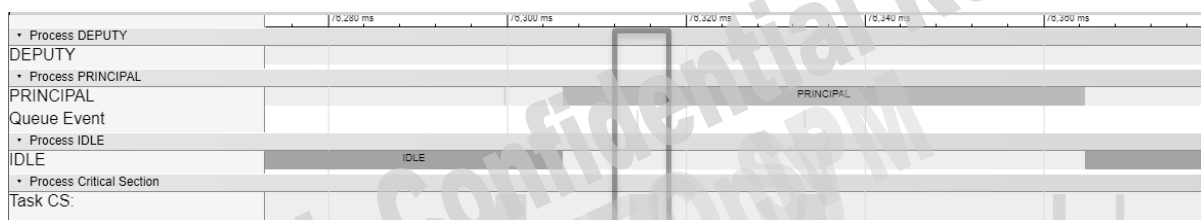


图 8-10. 软件定时器示例

– 临界区

- 通过对应的临界区向上，可以找到在临界区中运行的任务。



当任务“PRINCIPAL”正在运行时的临界区

图 8-11. 临界区示例

– 中断服务程序 (ISR)

- ISR 显示中断服务例程的执行时间和 ISR 编号。

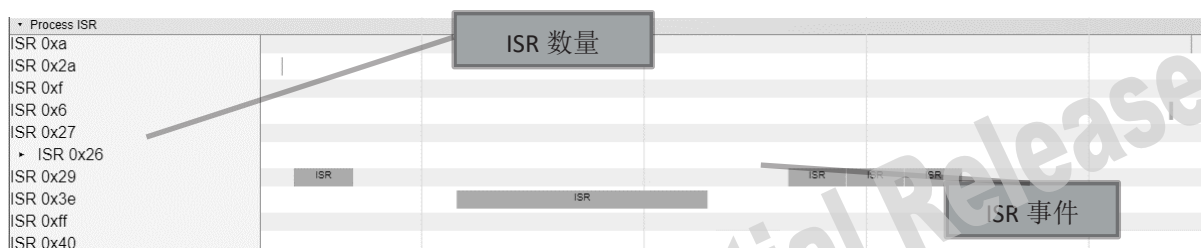


图 8-12. 中断服务程序示例

– OS 时钟节拍

- OS 时钟显示操作系统计时器滴答的时间。

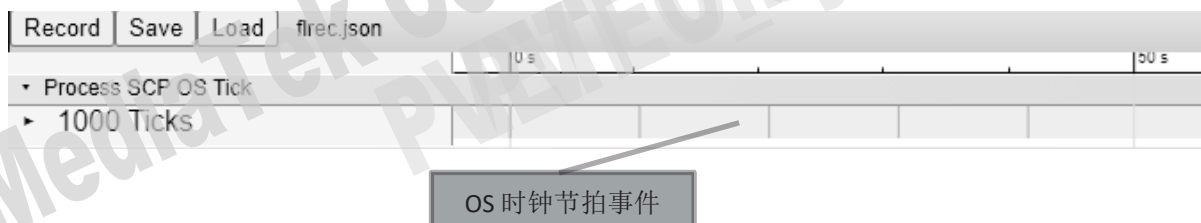


图 8-13. OS 时钟节拍示例

9 地址检测器

9.1 ASAN 使用方法

地址检测器（Address Sanitizer, ASAN），是一种地址健全性检查工具，用于发现对堆、栈和全局对象的越界访问，以及使用已释放内存的错误。

- 启用 ASAN
 - 设置配置并打开定义以启用 ASAN。
 - 路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/$Project/project.mk
master/src/tinysys/mt8676/scp/build/config.mk
```

表 9-1. 编译器选择和定义

#	编译器选项和定义	描述
1	CFG_ASAN_SUPPORT	Enable: yes Disable: no
2	-mllvm -asan-stack	Check local variable Enable: 1 Disable: 0
3	-mllvm -asan-global	Check global variable Enable: 1 Disable: 0
4	-mllvm -asan-memintrin	Check memset/memecpy Enable: 1 Disable: 0

- 异常类型

表 9-2. 异常类型表

#	异常类型	描述
1	ASAN_STACK_LEFT	局部变量下溢
2	ASAN_STACK_MID	局部变量下溢/上溢
3	ASAN_STACK_RIGHT	局部变量上溢
4	ASAN_STACK_PARTIAL	局部变量部分上溢
5	ASAN_GLOBAL_REDZONE	全局变量上溢

#	异常类型	描述
6	ASAN_HEAP_USE_AFTER_FREE	使用已释放的堆内存

问题处理：栈下溢

```
[14.504] (0) [0] KASAN detect stack-out-of-bounds at PC:0x15f52
[14.505] (0) [0] write 1Byte at [0x3c298] -> shadow [0x207853]
[14.506] (0) [0] Shadow bytes around the buggy address:
[14.507] (0) [0] 0x00207830: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.508] (0) [0] 0x00207840: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.509] (0) [0] =>0x00207850: f1 f1 04 [f2] 00 02 f3 f3 00 00 00 00 00 00 00 00
[14.510] (0) [0] 0x00207860: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.512] (0) [0] 0x00207870: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.513] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
00015f42 <asan_underflow_left>:
; asan_underflow_left():
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0_mpl-of_p8_b/vendor/mediatek
; {
15f42: 9005      push16, a0, m0
15f44: 717d      c.addi16sp sp, -16
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0_mpl-of_p8_b/vendor/mediatek
; {
15f46: fff50493  addi    s1, a0, -1 // 0xffff [12bitSE]
15f4a: 842a      c.mv    s0, a0
15f4c: 8526      c.mv    a0, s1
15f4e: 80b40ef  jal    -46932 <_asan_store1>
15f52: 8aa00513  li      a0, 170
```

```
attribute__((noinline)) void asan_underflow_left(char *ptr)
{
    ptr[-1] = 0xAA;
    printf("%s: underflow write %p\n", __func__, &ptr[-1]);
}
```

问题处理：全局变量上溢

```
[8.490] (0) [0] KASAN detect global-out-of-bounds at PC:0x15f64
[8.491] (0) [0] read 4Byte at [0x60288] -> shadow [0x20c051]
[8.492] (0) [0] Shadow bytes around the buggy address:
[8.493] (0) [0] 0x0020c030: f9 f9 f9 f9 00 f9 f9 f9 f9 f9 f9 04 f9 f9 f9
[8.494] (0) [0] 0x0020c040: f9 f9 f9 f9 00 f9 f9 f9 f9 f9 f9 00 00 00 00
[8.495] (0) [0] =>0x0020c050: 00 [f9] f9 f9 f9 f9 f9 f9 f9 f9 f9 f9 f9 f9 f9
[8.496] (0) [0] 0x0020c060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.498] (0) [0] 0x0020c070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.499] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
; /mfs/mtk/mt1157/mtk26942/casws/alps-mp-t0_mpl-of_p8_b/vendor/mediatek
; {
15f52: 00060537  lui    a0, 96 // 0x60000
15f56: 26050413  addi    s0, a0, 600 // 0x260 (i32: 0x60260)
15f5a: 02840493  addi    s1, s0, 40 // 0x28
15f5e: 8526      c.mv    a0, s1
15f60: 80b40ef  jal    -47094 <_asan_load4>
15f64: 834a      c.mv    a0, s2
```

```
attribute__((noinline)) void global_overflow(void)
{
    volatile uint32_t *ptr;
    volatile uint32_t test;
    ptr = &globalArr[0];
    test = ptr[10];
}
```

– 问题处理：使用已释放的堆内存

```
[9.230] (1) [0] KASAN detect heap use-after-free at [FC:0x40C2a]
[9.231] (1) [0] read 4Byte at [0x271b90] -> shadow [0x54e372]
[9.232] (1) [0] Shadow bytes around the buggy address:
[9.232] (1) [0] 0x0054e350: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[9.233] (1) [0] 0x0054e360: 00 00 00 00 00 00 00 00 00 00 00 fd fd fd fd
[9.234] (1) [0] =>0x0054e370: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.235] (1) [0] 0x0054e380: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] 0x0054e390: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 135
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
Heap use after free: fd
```

```
/mfs/mtkslt1157/mtk26942/casws/alps-mp-t0_mp1-of_p
if(ptr[5] == 0xAA)
40c22: 01440513 addi a0, s0, 20 // 0x14
40c26: 873cb0ef jal -214926 <__asan_load4>
40c2a: 4848 c.lw a0, 20(s0)
40d2c: 0aa00593 li a1, 170
40c30: 00b51863 bne a0, a1, 16 <.LBB9_2>
```

```
uint32_t *dynamicArr;
attribute__((noinline)) void asan_use_after_free(void)
{
    uint32_t *ptr;
    dynamicArr = pvPortMalloc(10*sizeof(uint32_t));
    vPortFree(dynamicArr);
    ptr = dynamicArr;
    if(ptr[5] == 0xAA)
        printf("Use after free\n");
}
```

9.2 如何将代码移动到 DRAM

启用 ASAN 后，代码大小将增加约 1.7 倍。因此，SRAM 大小可能不足。用户可以将某些功能或函数移动到 DRAM 以减少 SRAM 的使用。

• 使用属性

要在 DRAM 区域指定函数/变量，我们使用在 cache_internal 中定义的缓存宏。请注意，DRAM_REGION_VARIABLE 仅适用于全局/静态变量，因为局部变量使用栈作为存储。默认情况下，出于性能考虑，我们将栈保留在 SRAM 中。

```
#define DRAM_REGION_VARIABLE __attribute__((section(".dram_region_variable")))
#define DRAM_REGION_FUNCTION __attribute__((section(".dram_region_func")))
```

• 使用链接脚本

链接脚本描述了对象（即各个部分）在输出二进制文件中的映射方式。它提供了一种将对象定位到 DRAM 区域的简便方法。以下是一个模板头文件和链接脚本，用于将 C 文件中的所有部分定位到 DRAM 区域。

1. 首先，您需要检查 cache_ld.h 文件（路径：@project/RV55_A/\$platform/\$project），在这里您可以看到许多预定义的宏，格式为 CACHE_***_TEXT 和 CACHE_***_DATA。您可以添加一个新的宏，并自行命名。宏名后的字符串是期望的 C 文件路径。例如：（“?” 是通配符比较）

```
#define CACHE_BARO_TEXT \
    /*/middleware/contexthub/MEMS_Driver/barometer/?*.o(.text*) \
#define CACHE_BARO_DATA \
    /*/middleware/contexthub/MEMS_Driver/barometer/?*.o(.rodata* .data* .bss*) \
```

2. 您需要检查 `link.ld` 文件，它应该包含 `CACHE_***_TEXT` 和 `CACHE_***_DATA` 的响应

您可以添加 `CACHE_BARO_TEXT` 和 `CACHE_BARO_DATA` 后的字符串。例如：

```
dram_region __dram_start : AT(__dram_start) {  
#ifdef CACHE_BARO_TEXT  
    CACHE_BARO_TEXT  
#endif  
*(.dram_region_func)  
    *(.dram_region_ro)  
#ifdef CACHE_BARO_DATA  
    CACHE_BARO_DATA  
#endif  
}
```

10 附录

10.1 常见问题及解决方法

10.1.1 Malloc 失败

由于 SRAM 大小有限且较小，malloc 池大小是根据功能量身定制的，当引入新功能而不扩大池大小时，malloc 失败是常见的。

注意：

当发生 malloc 失败时，会显示一条失败消息：

```
"
[2.232](0) malloc fail
[2.232](0) [ASSERT] task: CHRE
"
```

此消息显示哪个任务 malloc 失败，随后会跟随核心转储日志。

• 如何解决

1. 修改 scp/project/RV55_A/Mt8676/platform/platform.mk, 扩大堆大小, 例如 80*1024

```
ifeq ($(CFG_CHRE_SUPPORT), yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (80 * 1024) ))) □ change 40
to 80
endif
```

2. 修改 scp/drivers/common/scpctl/scp_scpctl.c, 强制启用任务监控。

```
void scpctl_init(void)
{
...

    else { /* monitor task is in suspended state */
        scpctl.stat = SCPCTL_STAT_INACTIVE;
        scpctl.op = SCPCTL_OP_INACTIVE;
        //vTaskSuspend(xMonitorTask[id]); about line: 244, mark it
    }

...
}
```

3. 检查剩余堆大小并计算所需堆大小。

```
[210.009] (0) Heap: free/total: 40946/108544
```

4. 调整堆大小, 剩余堆大小为 40946 字节, $40946/1024 = 39.98\text{KB}$, 最小堆大小为 41KB, 设置为 43 或 44KB 更安全。请记得在问题修复后恢复 `scp/drivers/common/scpctl/scp_scpctl.c` 的更改。

```
ifeq ($(CFG_CHRE_SUPPORT),yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (43 * 1024) ))) )
endif
```

10.1.2 非对齐访问

SCP MDSP-RV33 处理器没有硬件来处理未对齐访问, 实施是通过软件完成的。当发生未对齐访问时, 处理器将引发异常并在处理程序中进行软件解决。因此, 性能成本显著。

注意:

当发生未对齐访问时, 会显示一条警告消息:

“Warning: MISALIGNED LOAD, pc:0x00001234, addr:0x00042232”

此消息显示有问题的 PC 和处理器要访问的地址。当未对齐访问持续发生时, 系统可能忙于打印此消息, 并可能导致超时断言

• 示例及解决方法

大多数未对齐访问是由于结构体应用了 `packed` 限定符。这可以通过移除它来避免。

- 示例:

```
struct pack_struct {
    unit32_t size;
    uint32_t crc;
    uint8_t type;
} __attribute__((packed));
```

当声明一个结构体数组时, 例如 `pack_struct st_array[10]`, 访问结构体成员将是未对齐的。另一种情况是使用整数指针访问字符数组。这可以通过添加 `__attribute__((aligned(4)))` 来避免。编译器不保证 `&char_array[0]` 是 4 字节对齐的。

- 示例:

```
uint8_t char_array[64];
test_value = *(uint32_t *)(&char_array[0])
```

10.2 代码大小限制

Mt86768676 总 SRAM 大小为 2MB，实际的 SCP SRAM 大小可以通过检查符号 `_end` 来确定。

剩余的 SRAM 大小：

```
Mt8676: 0x300000 - _end
```

在 Mt8676 中，我们可以在 `project/RV55_A/Mt86768676/platform/link.ld.c` 文件中将 SRAM 区域长度设置为 2MB

示例：`sram : ORIGIN = 0x00000000, LENGTH = 0x00200000.`

当大小超过 2MB 限制时，这将导致构建失败。

代码大小检查工具将在每次构建时运行并输出详细信息。检查图 10-1，垂直是项目名称，水平是每个项目的代码大小。例如，CHRE 的总大小为 62985（总和）字节，其中 `.text` 部分占总大小的 35598 字节。

	. = ALIGN (...sync	.text	Sum
C-lib	0 ... 0	0	
CHRE	0 ...	35598	62985
DSP	0 ... 0	0	
DVFS	0 ... 7574	8606	
Heap	0 ... 334	82275	
Peripheral	0 ... 18064	26981	
Platform	256 ... 66300	313481	
RTOS	0 ... 13776	14536	
Sensor	0 ... 0	0	
VOW	0 ... 0	0	

图 10-1. 内存工具报告

10.2.1 代码大小检查工具的使用

工具 `memoryReport.py` 是一个脚本，用于在构建时限制代码大小。如果代码大小超过您的设置，它将导致构建错误。

- 路径

```
master/src/tinysys/mt8676/common/tools/memoryReport.py
```

- 设置配置文件路径

```
master/src/tinysys/mt8676/scp/project/RV55_A/Mt8676/platform/Setting.ini
```

- 配置文件格式（`setting.ini`）

```
[TinySys-SCP]
$File_Name: $Main_feature: $Sub_feature
[SCP-MT6985]
$Main_feature : Max_code_size
$Sub_feature : Max_code_size
-----
* File name: Full file path or Partial file path (Ex:middleware/contexthub/perf)
* Main feature, (Ex: Sensor, Audio), the main feature that this file belongs to
```

```
* Sub feature, (Ex: gyro, pedometer), the sub feature that this file belongs to
* Main_feature/Sub_feature (after SCP-MT6985):
  - Main or Sub feature maximum size limit
```

- 内存检查失败

- 参考示例:

```
SCP: I2C(3958>110) is out of memory limitation
SCP: SPI(6316>1100) is out of memory limitation
make: *** [tinysys_out/RV55_A/scp/tinysys-scp-RV55_A.elf] Error 13
```

10.3 Scp_Region_Info 架构

struct scp_region_info_st 是一个指向 SCP SRAM 中固定地址的指针，用于在 IPI 准备好之前从引导加载程序向 SCP 传递参数。如果添加了新的成员，记得同步引导加载程序 (lk2), 内核(kernel-6.1) 和 SCP 仓库代码。如果结构在仓库之间不同步，SCP 可能会启动失败。

- LK 头文件路径:

```
yocto/src/bsp/lk2/platform/mediatek/mt6897/common/scp/scp_plat_priv.h
```

- Kernel 头文件路径:

```
master/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/misc/mediatek/scp/rv/scp_helper.h
```

- SCP 头文件路径:

```
master/src/tinysys/mt8676/scp/project/RV55_A/mt6897/platform/inc/main.h
```

- SCP 头文件路径:

```
master/src/tinysys/mt8676/scp/project/RV55_A/mt6897 /platform/boot55.S
```

scp_region_info 也在 boot55.S 中定义。

```
scp_region_info:
.long    0x00000000    /* 0x04 ap_loader_start */
.long    0x00000000    /* 0x08 ap_loader_size */
...
.long    0x00100000    /* 0x34 regdump size */
.long    0x00000000    /* 0x38 param start address */
.long    0x00000000    /* 0x3c ap_params_start */
.long    0x12345678    /* 0x40 a test ID */
```

例如，如果在结构 scp_region_info 中添加了一个名为 test ID 的新成员，必须在 boot55.S 中添加默认值。

在 SCP 端访问 scp_region_info_st 时，必须将其分配给 NULLPTR_BASE，例如：

```
void scpctl_init(void)
{
    int ret = 0;
#ifdef CFG_NULLPTR_TRAP
    struct scp_region_info_st region_info = *(struct scp_region_info_st
*) (NULLPTR_BASE + 0x4);
```

#endif

10.4 SCP 恢复机制

当 SCP 崩溃或没有响应时，Linux SCP 驱动程序将执行 SCP 恢复。在恢复过程中，SCP 将恢复正常。但我們需要注意以下几点：

- 在 SCP 重置时，SCP 驱动程序将清除 SCP SRAM/DRAM 内容，重置 RV55 处理器在 ATF，并重新执行启动流程。（清除所有程序文本、bss 和数据段）
- SCP 仅重置处理器，而不重置外设（如传感器、I2C 模块和其他设备），因此如果有必要，外设驱动程序应在初始阶段自行重置。
- Linux SCP 驱动程序 API 的开发人员必须遵循 10.4.2 章节，并确保在恢复期间 Linux 驱动程序与 SCP 之间没有任何通信。
- 当 SCP 重新启动时，Linux SCP 驱动程序将重新初始化。与 SCP 驱动程序相关的驱动程序必须确保重新启动流程不会影响其功能。

10.4.1 SCP 恢复流程

当 SCP 异常或一段时间没有响应时，恢复将开始。SCP 内核驱动程序和 SCP 将进入服务中断状态。在此期间，SCP 内核驱动程序将重置 SCP 并向所有已注册通知链的驱动程序发送 `SCP_EVENT_STOP`。当 SCP 恢复正常后，SCP 内核驱动程序将向所有驱动程序发送 `SCP_EVENT_READY`。

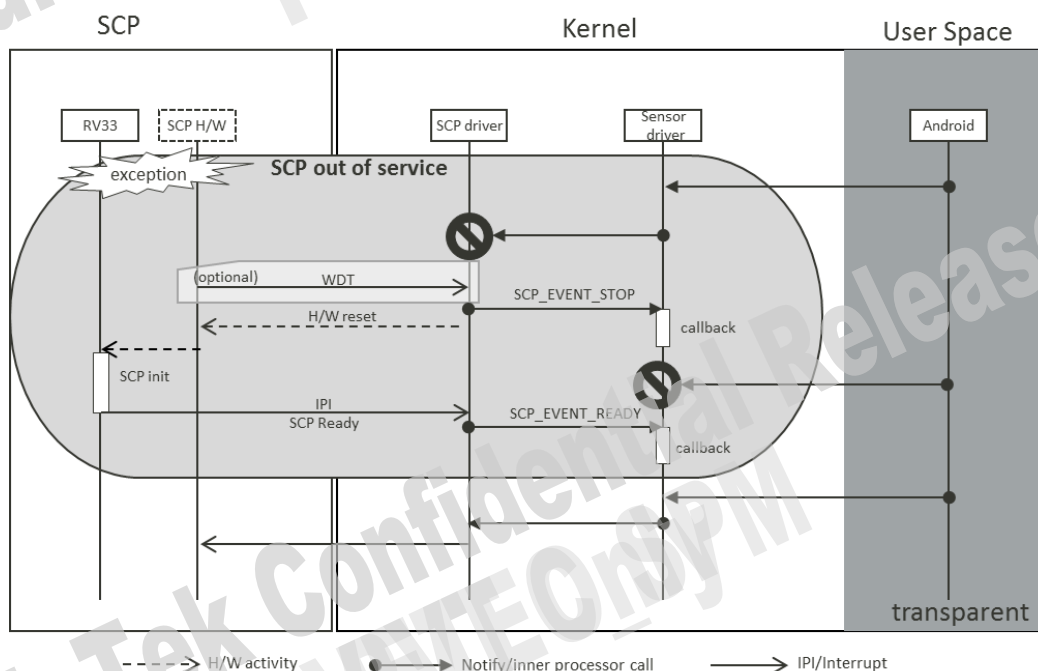


图10-2. SCP 重置流程

10.4.2 恢复通知流程

Linux SCP 驱动程序的开发人员必须遵循此 SCP 事件通知链，以确保 SCP 恢复有效。

- API: 接收通知并调用已注册的回调函数

```
void scp_A_register_notify(struct notifier_block *nb)
description
    注册通知链的回调函数。
parameters
    nb: 通知链的回调函数
callback parameters
    SCP_EVENT_READY: 当 SCP 初始化完成时，开始与 SCP 对应的任务
    SCP_EVENT_STOP: 当 SCP 即将重置时，停止与 SCP 对应的任务。
```

注意:
所有回调函数可能会被多次调用，并且绝对不能被阻塞。

- 示例:
 - 需要包含头文件 <linux/notifier.h>, <mach/scp_helper.h>
 - 调用 scp_register_notify () 时，使用参数: SCP_EVENT_READY or SCP_EVENT_STOP

```
static void task_start(void) {
    // start tasks
}
static void task_stop(void) {
    // stop tasks
}
static int app_event(struct notifier_block *this, unsigned long event, void *ptr) {
    switch (event) {
        case SCP_EVENT_READY:
            task_start();
            break;
        case SCP_EVENT_STOP:
            task_stop();
            break;
    }
    return NOTIFY_DONE;
}
static struct notifier_block app_notifier = {
    .notifier_call = app_event,
};
static int __init scp_app_init(void) {
    if (scp_is_ready()) {
        task_start();
    }
}
```

```
}  
scp_register_notify(&app_notifier);  
}
```

注意:

1. 必须注册通知链，因为在恢复期间 `scp_ipi_send()` 可能会返回错误。
2. 必须应用错误处理流程：
 - 在接收到 `SCP_EVENT_STOP` 后立即停止调用 `scp_ipi_send()`
 - 在接收到 `SCP_EVENT_READY` 后恢复调用 `scp_ipi_send()`

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁灭所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。